

Machine Learning, Assignment 2

Date posted: Nov. 21, 2011,

Due: Nov. 28, 2011 at the *start* of class

1. (4 points) Consider a neural network with K output units y_k that are combined using a softmax activation function to yield class-probabilities:

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{\exp(y_k(\mathbf{x}))}{\sum_j \exp(y_j(\mathbf{x}))}$$

Compute the derivative of the log-probability $\sum_n \log p(t_n|\mathbf{x}_n)$ of training data with respect to some unit y_k . Discuss why this derivative is needed when learning network parameters with backprop.

2. (6 points) Download the data files

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/mnist_train_images_subset.npy`

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/mnist_train_labels_subset.npy`

and the test data files

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/mnist_test_images_subset.npy`

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/mnist_test_labels_subset.npy`

and save them locally. The files contain 1000 binary train and test images of handwritten digits and corresponding class-labels. Each image is represented as a 784-dimensional binary vector. There are 10 classes, labeled using integers $0, \dots, 9$.

Estimate a Bernoulli Naive Bayes classifier from the training data, and apply the classifier on both the training and the test data.

What to hand in:

- A plot of the class prior probabilities. You may use the `pylab.bar()` function for this end.
- Ten images of size 28×28 pixels, each showing the class-conditional Bernoulli parameters for one class, using gray values.
- Two numbers: The percentage of correctly classified training cases and the percentage of correctly classified test cases. (– SEE NEXT PAGE FOR HINTS –)

Hints:

- To learn the naive Bayes classifier, you will need to estimate prior probabilities over classes as well as class-*conditional* Bernoulli probabilities over the 784-dimensional image vectors.
- You may want to represent all probabilities using their log's when doing any calculations with them, to avoid underflow.
- To avoid taking the log of 0.0 you can do the following: When estimating the class-conditional probabilities, assume your training dataset contains *two* extra images: One “all-white” image consisting only of ones, and one “all-black” image consisting only of zeros. (This means, you assume the total number of images in the training set is 1002 not 1000.) Alternatively, you can add small numbers to your probability estimates.
- To visualize images you can use `pylab.imshow()`. Reshape the 784-dimensional vectors into matrices of size 28×28 pixels before passing them to `imshow()`. You can use `pylab.gray()` to set the colormap so images are represented with gray values.